

Semester Project Proposal

Project Title: SmartGradeAI

Group Information

Section: SPM-A

1. **Bilal Javed** (Project Captain) 22i-0788
2. **Hasan Ali Syed** 22i-0919
3. **Ammar Ahmad** 22i-1267

1. Problem Statement

Manual grading of descriptive answers is a labor-intensive, time-consuming process that is often prone to subjectivity and inconsistency across different evaluators. As class sizes increase, instructors face significant challenges in maintaining grading quality and speed. There is a critical need for an intelligent system that can assist educators by evaluating student answers objectively, ensuring consistency based on predefined rubrics and model solutions.

2. Proposed Solution

We propose an **Automated Answer Evaluation System** utilizing Large Language Models (LLMs) and Retrieval Augmented Generation (RAG). This system aims to streamline the grading process by automatically assessing student responses against teacher-provided criteria. The workflow involves:

- **Input:** Teachers upload exam questions and detailed grading rubrics.
- **Processing:** The system generates a model reference solution and stores it alongside the rubric in a vector database.
- **Evaluation:** When a student submits an answer, the system retrieves the relevant reference material using RAG and compares the semantic meaning of the student's answer against the model solution.
- **Output:** The system generates a score and feedback based on alignment with the rubric.

3. Functional Requirements

3.1 Teacher Module

- **Exam Management:** Interface to upload exam question papers.
- **Rubric Management:** Capability to upload and define grading rubrics (criteria and marks distribution).
- **Solution Generation:** Trigger the LLM to generate model/reference solutions for the uploaded questions.
- **Vectorization:** Functionality to convert and store all reference documents as vector embeddings.
- **Result Review:** Dashboard to view evaluated results, scores, and system-generated feedback.

3.2 Student Module

- **Submission Portal:** Interface for students to upload their descriptive answers for a specific exam ID.
- **Result Viewing:** (Optional) Interface to view awarded marks and feedback after the grading process is complete.

3.3 AI Grading Module

- **Embedding Generation:** Convert exams, rubrics, and reference solutions into high-dimensional vector embeddings.
- **Storage:** Store embeddings efficiently in a vector database.
- **Retrieval (RAG):** Retrieve relevant reference content contextually matching the student's answer.
- **Semantic Evaluation:** Compare student answers with reference solutions using semantic similarity measures.
- **Scoring Engine:** Assign scores strictly based on the rubric alignment and similarity thresholds.

3.4 Data Storage

- **Vector Database:** Store document chunks and embeddings.
- **Relational/Metadata Store:** Maintain metadata including Exam IDs, Question Numbers, Student IDs, and Max Marks.

4. Non-Functional Requirements

- **Accuracy:** The output must align with the criteria defined in the rubric to ensure fairness.
- **Consistency:** The system must be deterministic; the same answer should receive similar scores across multiple evaluation runs.
- **Scalability:** The architecture must support concurrent uploads for multiple students and exams.
- **Security:** Role-based access control (RBAC) to ensuring only authorized users (Teachers) can upload rubrics or view all student grades.
- **Usability:** The interface must be intuitive, allowing for simple document uploads and clear results.

5. Key Use Cases

Use Case ID	Actor	Description	Outcome
UC-01	Teacher	Uploads exam questions and grading rubric.	Documents are stored and converted into vector embeddings.
UC-02	Teacher / System	System uses LLM to generate a model solution based on the exam context.	Reference solution is stored in the vector database.
UC-03	Student	Student uploads their exam answers.	Answer is stored and queued for evaluation.
UC-04	System	System retrieves context via RAG, compares semantics, and computes rubric score.	Marks are assigned automatically to the student record.
UC-05	Teacher	View marks and AI-generated feedback.	Grading process is finalized.

6. Technology Stack

- **LLM:** GPT-4o / Llama 3 (via API or local hosting)
- **Vector Database:** Pinecone / ChromaDB / FAISS
- **Backend Framework:** Python (FastAPI or Flask)
- **Embeddings:** OpenAI Embeddings / HuggingFace Sentence Transformers
- **Frontend:** React / Streamlit (for prototype interface)

7. Project Timeline (Tentative)

Phase 1: Planning & Analysis (Weeks 1-2)

- Conduct literature review on RAG architecture and LLM integration.
- Finalize functional requirements and analyze system constraints.
- Define specific grading rubrics and scoring logic.
- **Deliverables:** Project Proposal, SRS Document.

Phase 2: System Design (Weeks 3-5)

- Design overall system architecture and database schema.

- Select appropriate Vector Database and LLM API providers.
- Create UI wireframes for both Teacher and Student dashboards.
- **Deliverables:** System Design Document (SDD), UI Prototypes.

Phase 3: Core Development (Weeks 6-9)

- Set up the Vector Database environment (e.g., Pinecone or Chroma).
- Implement backend logic for document parsing and embedding generation.
- Develop the core RAG retrieval pipeline for context matching.
- **Deliverables:** Functional Backend API, Vector Store Setup.

Phase 4: Integration & UI (Weeks 10-12)

- Develop the Frontend interfaces for Teacher and Student modules.
- Integrate the backend LLM grading logic with the user interface.
- Implement file upload handling and result visualization features.
- **Deliverables:** Integrated System Prototype.

Phase 5: Testing & Refinement (Weeks 13-14)

- Test grading accuracy by comparing AI results against manual grading examples.
- Optimize RAG retrieval parameters to ensure correct context is fetched.
- Refine prompt engineering to improve feedback quality.
- **Deliverables:** Testing Report, Optimized Model.

Phase 6: Final Submission (Weeks 15-16)

- Compile final documentation and user manuals.
- Prepare project presentation and live demonstration.
- **Deliverables:** Final Project Code, Final Report.

8. Task Division

Bilal: Project Lead (Captain)

- **System Architecture:** Designing the overall RAG workflow and component interaction.
- **AI Core Implementation:** Developing the LLM logic, Prompt Engineering, and Semantic Similarity algorithms.
- **Integration:** Merging backend services with the AI module and overseeing final deployment.

Hasan: Backend & Database Engineer

- **Vector Database Setup:** Managing embeddings storage (e.g., Pinecone/Chroma) and retrieval efficiency.
- **API Development:** Building RESTful APIs (FastAPI/Flask) to handle requests between frontend and AI services.
- **Data Processing:** Implementing logic to parse and chunk uploaded PDF documents (exams/rubrics).

Ammar: Frontend & QA Engineer

- **UI/UX Development:** Building the Teacher and Student dashboards (React/Streamlit).
- **Security:** Implementing user authentication and role-based access control.
- **Testing & Documentation:** Conducting accuracy tests on grading results and compiling the final report/user manual.